

HiDiTingV100 设备驱动

开发指南

文档版本 00B01

发布日期 2025-06-05

前言

概述

本文档主要介绍 HiDiTingV100 设备驱动开发的相关内容，主要包括工作原理、按场景描述接使用方法和注意事项。用于指导用户快速使用设备驱动接口。

产品版本

与本文档相对应的产品版本如下。

产品名称	产品版本
HiDiTing	V100

读者对象

本文档主要适用于以下工程师：

- 技术支持工程师
- 软件开发工程师

符号约定

在本文中可能出现下列标志，它们所代表的含义如下。

符号	说明
----	----

符号	说明
 危险	表示如不可避免则将会导致死亡或严重伤害的具有高等级风险的危害。
 警告	表示如不可避免则可能导致死亡或严重伤害的具有中等级风险的危害。
 注意	表示如不可避免则可能导致轻微或中度伤害的具有低等级风险的危害。
须知	用于传递设备或环境安全警示信息。如不可避免则可能会导致设备损坏、数据丢失、设备性能降低或其它不可预知的结果。 “须知”不涉及人身伤害。
 说明	对正文中重点信息的补充说明。 “说明”不是安全警示信息，不涉及人身、设备及环境伤害信息。

修改记录

文档版本	发布日期	修改说明
00B01	2025-06-05	第一次临时版本发布。

目 录

前言.....i

1 UART.....1

1.1 概述..... 1

1.2 功能描述..... 3

1.3 开发指引..... 4

1.4 注意事项..... 8

2 SPI.....9

2.1 概述..... 9

2.2 功能描述..... 9

2.3 开发指引..... 10

2.4 注意事项..... 15

3 I2C.....17

3.1 概述..... 17

3.2 功能描述..... 17

3.3 开发指引..... 18

3.4 注意事项..... 20

4 ADC.....22

4.1 概述..... 22

4.2 功能描述..... 22

4.3 开发指引..... 23

4.3.1 ADC 手动采样开发指引 23

4.3.2 ADC 自动采样开发指引 26

4.4 注意事项..... 28

5 Pinctrl/GPIO.....29

5.1 概述.....	29
5.2 功能描述.....	29
5.3 开发指引.....	30
5.4 注意事项.....	32
6 DMA.....	33
6.1 概述.....	33
6.2 功能描述.....	33
6.3 开发指引.....	34
6.4 注意事项.....	36
7 PWM.....	37
7.1 概述.....	37
7.2 功能描述.....	37
7.3 开发指引.....	38
7.4 注意事项.....	39
8 Watchdog.....	40
8.1 概述.....	40
8.2 功能描述.....	40
8.3 开发指引.....	41
8.4 注意事项.....	41
9 Timer.....	43
9.1 概述.....	43
9.2 功能描述.....	43
9.3 开发指引.....	44
9.4 注意事项.....	44
10 RTC.....	46
10.1 概述.....	46
10.2 功能描述.....	46
10.3 开发指引.....	47
10.4 注意事项.....	47
11 Systick.....	48
11.1 概述.....	48

11.2 功能描述.....	48
11.3 开发指引.....	49
11.4 注意事项.....	49
12 TCXO.....	50
12.1 概述.....	50
12.2 功能描述.....	50
12.3 开发指引.....	51
12.4 注意事项.....	51
13 Calendar	52
13.1 概述.....	52
13.2 功能描述.....	52
13.3 开发指引.....	53
13.4 注意事项.....	54
14 Flash	55
14.1 概述.....	55
14.2 功能描述.....	55
14.3 开发指引.....	56
14.4 注意事项.....	58
15 Key	59
15.1 概述.....	59
15.2 功能描述.....	59
15.3 开发指引.....	60
15.4 注意事项.....	61
16 SDIO	63
16.1 概述.....	63
16.2 功能描述.....	63
16.3 开发指引.....	64
16.4 注意事项.....	65

插图目录

图 1-1 UART 协议帧格式..... 2

1 UART

1.1 概述

1.2 功能描述

1.3 开发指引

1.4 注意事项

1.1 概述

UART (Universal Asynchronous Receiver/Transmitter)，通用异步收发传输器。

UART 是一种通用串行数据总线，用于异步通信。该总线双向通信，可以实现全双工传输和接收。

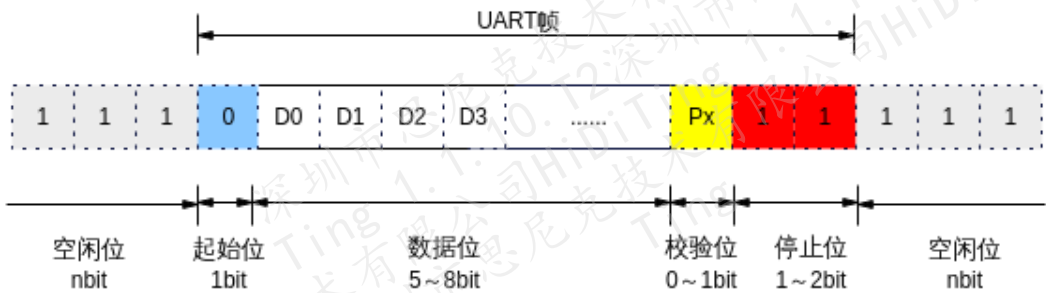
- 串行：指在计算机总线或其他数据通道上，每次传输一个位元数据，并连续进行以上单次过程的通信方式。
- 异步：无时钟信号，收发双方需要约定使用相同的时钟频率，需要起始位和停止位。
- 全双工：通信允许数据在两个方向上同时传输，它在能力上相当于两个单工通信方式的结合。全双工可以同时进行信号的双向传输。

芯片提供 2 个 UART 单元：UART0（用于 A 核串口日志打印与 AT 命令）、UART1（用于与电脑上 debugkit 软件与镜像资源的烧录）。

在 config.py 对应的编译的 target 里面添加宏 FT_SINGE_UART，那么 UART0 的功能将复用到 UART1 上，UART0 可以用来对接其他外设比如 GPS 芯片、CAT1 芯片。

UART 协议帧格式如图 1-1 所示。

图1-1 UART 协议帧格式



- 空闲位：处于逻辑 1 状态，表示当前线路上没有资料传送。
- 起始位：先发出一个逻辑 0 信号，表示传输数据的开始。
- 数据位：可以是 5 ~ 8 位逻辑 0 或 1。如 ASCII 码（7 位），扩展 BCD 码（8 位）小端传输。
- 奇偶校验位：
数据位传送完成后，可以选择是否进行奇偶校验（收发双方约定），串口校验分以下几种方式：
 - 无校验（no parity）。
 - 奇校验（odd parity）：如果数据位中 ‘1’ 的数目是偶数，则校验位为 ‘1’；如果 ‘1’ 的数目是奇数，校验位为 ‘0’。
 - 偶校验（even parity）：如果数据为中 ‘1’ 的数目是偶数，则校验位为 ‘0’；如果为奇数，校验位为 ‘1’。
 - mark parity：校验位始终为 1。
 - space parity：校验位始终为 0。
- 停止位：数据结束标志，可以是 1 位、1.5 位、2 位的高电平，我们的芯片默认使用是 1 位的。

波特率计算公式

UART 的波特率=内部总线频率/(16×分频系数)

假设：UART_FBRD=0x1E、UART_IBRD=0x01，这就表示分频系数的整数部分 30，小数部分为 0.015625，整个分频系数为 30.015625。

则有：UART 的波特率=内部总线频率/(16×分频系数)=内部总线频率/(16×30.015625)

如：UART3 内部总线时钟为 81M，分频系数最小为 1，所以波特率最高可以配置为：81M/(16×1)=5.06M。（恰好满足规格要求的 5M）

UART 流控制

当接收端的数据缓冲区已满，无法处理数据时，就发出“不再接收”的信号，发送端则停止发送，直到发送端收到“可以继续发送”的信号再发送数据。

计算机中常用的两种流控制分别是硬件流控制（RTS /CTS、DTR /DSR 等）和软件流控制（XON /XOFF）。

硬件流控制原理

UART 支持 4 线模式（4 线模式即 TX、RX、CTS、RTS）。

在 4 线工作模式下，UART RX FIFO 达到设定的中断阈值，则把 RTS 管脚电平拉高，相当于输出“不再接收”的信号。

在 4 线工作模式下，UART CTS 管脚被拉高，则 UART 无法通过 TX 发送数据，相当于输入了“不可继续发送”的信号。

1.2 功能描述

说明

若 UART 驱动需要支持 DMA 数据收发，需确保 DMA 驱动已完成初始化。

UART 模块提供的接口：

- uapi_uart_init：初始化 UART。
- uapi_uart_deinit：去初始化 UART。
- uapi_uart_read：读数据。
- uapi_uart_write：写数据。
- uapi_uart_set_flow_ctrl：配置 UART 硬流控。
- uapi_uart_set_software_flow_ctrl_level：配置软件流控的等级。
- uapi_uart_get_attr：获取 UART 配置参数。
- uapi_uart_set_attr：设置 UART 配置参数。
- uapi_uart_has_pending_transmissions：查询 UART 是否正在传输数据。
- uapi_uart_register_rx_callback：注册接收回调函数，此回调函数会根据触发条件和 Size 触发。
- uapi_uart_register_parity_error_callback：注册奇偶校验错误处理的回调函数。
- uapi_uart_register_frame_error_callback：注册帧错误处理回调函数。

- uapi_uart_write_int: 使用中断模式将数据发送到已打开的 UART 上, 当数据发送完成, 会调用回调函数。
- uapi_uart_write_by_dma: 通过 DMA 发送数据。
- uapi_uart_flush_rx_data: 刷新 UART 接收 Buffer 中的数据。
- uapi_uart_get_rx_data_count: 获取当前接收 Buffer 中的数据。
- uapi_uart_read_by_dma: 通过 DMA 读数据。

1.3 开发指引

以应用 UART0 为例, 数据收发流程如下:

步骤 1 UART 初始化。配置 UART 的波特率、数据位等属性, 并使能 UART。

```
unsigned char g_uart_rx_buff[TEST_UART_RX_BUFF_SIZE] = { 0 };
uart_buffer_config_t g_uart_buffer_config = {
    .rx_buffer = g_uart_rx_buff,
    .rx_buffer_size = TEST_UART_RX_BUFF_SIZE
};
errcode_t usr_uart_init_config()
{
    errcode_t errcode;
    uart_attr_t attr = {
        .baud_rate = 115200, /* 波特率 */
        .data_bits = UART_DATA_BIT_8, /* 数据位 */
        .stop_bits = UART_STOP_BIT_1, /* 停止位 */
        .parity = UART_PARITY_NONE, /* 校验位 */
    };
    uart_pin_config_t pin_config = {
        .tx_pin = S_MGPIO19, /* uart0 tx 配置IO复用 */
        .rx_pin = S_MGPIO20, /* uart0 rx 配置IO复用 */
        .cts_pin = PIN_NONE, /* 流控功能, 可选, 不建议使用 */
        .rts_pin = PIN_NONE /* 流控功能, 可选, 不建议使用 */
    };
    uart_extra_attr_t extra_attr = {
        .tx_dma_enable = tx_dma_enable,
        .tx_int_threshold = 0,
    };
```

```
.rx_dma_enable = rx_dma_enable,
.rx_int_threshold = 0
};
errcode_t errcode = uapi_uart_init(UART_BUS_1, &pin_config, &attr, &extra_attr,
&g_uart_buffer_config);
if (errcode != ERRCODE_SUCC) {
    printf("uart init fail\n");
}
return errcode;
}
```

步骤 2 UART 数据收发。调用 UART 读写数据接口，进行数据收发。

```
// 最常用的读取数据方式，注册rx中断回调，在回调中读取数据
void callback(const void *buffer, uint16_t length, bool error){
    /*buffer为rx接受到的数据，length为RX接收到的数据长度，error接收到的数据是否正确，当
error为1时代表数据波形错误，无效无数
};
uapi_uart_register_rx_callback(UART_BUS_1, UART_RX_CONDITION_FULL_OR_IDLE, 1024,
callback);
void usr_uart_read_data()
{
    // 基本不会使用这个方式读取RX数据
    int len;
    unsigned char g_test_uart_rx_buffer[64];
    len = uapi_uart_read(UART_BUS_0, g_test_uart_rx_buffer, 64, 0);
    if(len > 0) {
        /* process */
    }
}
int usr_uart_write_data(unsigned int size, char* buff)
{
    // 通过UART的TX写数据
    unsigned char tx_buff[TEST_UART_TX_BUFF_SIZE] = { 0 };
    if (memcpy_s(tx_buff, TEST_UART_TX_BUFF_SIZE, buff, size) != OK) {
        return ERROR;
    }
    int ret = uapi_uart_write(UART_BUS_0, tx_buff, size, 0);
    if(ret == -1) {
        return ERROR;
    }
}
```

```

    }
    return OK;
}

```

----结束

UART DMA 模式发送数据流程如下。

步骤 1 DMA 和 UART 初始化。配置 UART 的波特率、数据位等属性，并使能 UART。

```

uapi_dma_init();
uapi_dma_open();
errcode_t usr_uart_init_config()
{
    errcode_t errcode;
    uart_attr_t attr = {
        .baud_rate = 115200, /* 波特率 */
        .data_bits = UART_DATA_BIT_8, /* 数据位 */
        .stop_bits = UART_STOP_BIT_1, /* 停止位 */
        .parity = UART_PARITY_NONE, /* 校验位 */
    };
    uart_pin_config_t pin_config = {
        .tx_pin = S_MGPIOP19, /* uart0 tx 配置IO复用 */
        .rx_pin = S_MGPIOP20, /* uart0 rx 配置IO复用 */
        .cts_pin = PIN_NONE, /* 流控功能, 可选, 不建议使用 */
        .rts_pin = PIN_NONE /* 流控功能, 可选, 不建议使用 */
    };
    errcode_t errcode = uapi_uart_init(UART_BUS_0, &pin_config, &attr, NULL,
    &g_uart_buffer_config);
    if (errcode != ERRCODE_SUCC) {
        printf("uart init fail\n");
    }
    return errcode;
}

```

步骤 2 UART DMA 数据发。

```

#define TEST_UART_DMA_SEND_BUFF_SIZE 1024
#define HAL_DMA_TRANSFER_WIDTH_8 0
#define HAL_DMA_BURST_TRANSACTION_LENGTH_4 1
static errcode_t test_uart_write_by_dma()

```

```

{
    uint8_t dma_buff[TEST_UART_DMA_SEND_BUFF_SIZE] = { 0 };
    if (memset_s(dma_buff, TEST_UART_DMA_SEND_BUFF_SIZE, 0xA5,
TEST_UART_DMA_SEND_BUFF_SIZE) != 0) {
        return ERRCODE_FAIL;
    }
    uart_write_dma_config_t dma_cfg = {
        .src_width = HAL_DMA_TRANSFER_WIDTH_8,          /* 0代表8bit */
        .dest_width = HAL_DMA_TRANSFER_WIDTH_8,         /* 0代表8bit */
        .burst_length = HAL_DMA_BURST_TRANSACTION_LENGTH_4, /* 代表4字节 */
        .priority = 0                                    /* 优先级0 */
    };
    if (uapi_uart_write_by_dma(UART_BUS_0, dma_buff, len, &dma_cfg) != len) {
        printf("[UART] *** memory to uart fail!\n");
        return ERRCODE_FAIL;
    }
    return ERRCODE_SUCC;
}

```

步骤3 UART DMA 数据收。

```

#define TEST_UART_DMA_SEND_BUFF_SIZE 1024
#define HAL_DMA_TRANSFER_WIDTH_8 0
#define HAL_DMA_BURST_TRANSACTION_LENGTH_4 1
static errcode_t test_uart_read_by_dma()
{
    uint8_t dma_buff[TEST_UART_DMA_SEND_BUFF_SIZE] = { 0 };
    if (memset_s(dma_buff, TEST_UART_DMA_SEND_BUFF_SIZE, 0xA5,
TEST_UART_DMA_SEND_BUFF_SIZE) != 0) {
        return ERRCODE_FAIL;
    }
    uart_read_dma_config_t dma_cfg = {
        .src_width = HAL_DMA_TRANSFER_WIDTH_8,          /* 0代表8bit */
        .dest_width = HAL_DMA_TRANSFER_WIDTH_8,         /* 0代表8bit */
        .burst_length = HAL_DMA_BURST_TRANSACTION_LENGTH_4, /* 代表4字节 */
        .priority = 0                                    /* 优先级0 */
    };
    if (uapi_uart_read_by_dma(UART_BUS_0, dma_buff, len, &dma_cfg) != len) {
        printf("[UART] *** memory to uart fail!\n");
        return ERRCODE_FAIL;
    }
}

```

```
}  
    return ERRCODE_SUCC;  
}
```

----结束

1.4 注意事项

- SDK 中, UART1 默认用作程序烧写和维测数据通道。
- SDK 中, UART0 默认用于 BT 认证和 WIFI 共享。

2 SPI

- 2.1 概述
- 2.2 功能描述
- 2.3 开发指引
- 2.4 注意事项

2.1 概述

SPI (Serial Peripheral Interface) 可作为 Master 或 Slave 与外部设备进行同步串行通信 (外围设备必须支持 SPI 帧格式)。每个 SPI 具有收发分开的位宽为 8bit×32 的 FIFO。

2.2 功能描述

说明

SPI 主模式支持轮询模式读写、中断模式读写以及 DMA 模式读写；从模式也支持轮询模式读写、支持中断模式读写和 DMA 模式读写。

如果 SPI 驱动想配置 DMA 模式读写数据，需要确保 DMA 驱动已经初始化。DMA 初始化请参见“6 DMA”进行配置。

SPI 支持手动模式和自动模式。

- 手动模式：支持轮询模式读写、中断模式读写以及 DMA 模式读写。

CONFIG_SPI_SUPPORT_DMA：支持 DMA。

CONFIG_SPI_SUPPORT_INTERRUPT：支持中断。

- 自动模式：支持 DMA 模式和轮询模式自动切换。SPI 自动模式配置宏定义后，按照轮询的读写方式即可。

CONFIG_SPI_SUPPORT_DMA：支持 DMA。

CONFIG_SPI_SUPPORT_POLL_AND_DMA_AUTO_SWITCH：支持自动识别传输模式。

CONFIG_SPI_AUTO_SWITCH_DMA_THRESHOLD=0x8：自动识别传输阈值，大于阈值使用 DMA 模式，小于等于阈值使用轮询模式，阈值大小可调。

SPI 模块提供的接口：

- uapi_spi_init：初始化 SPI（包括：主从设备、极性、相性、帧协议、传输频率、传输位宽等设定）。
- uapi_spi_deinit：去初始化 SPI（关闭相应的 SPI 单元，释放资源）。
- uapi_spi_get_attr：获取 SPI 的基础配置参数（主从模式、时钟极性、时钟相位、时钟分频系数、SPI 工作频率、串行传输协议、SPI 帧格式、SPI 帧长度、SPI 传输模式等）。
- uapi_spi_set_attr：设置 SPI 的基础配置参数。
- uapi_spi_get_extra_attr：获取 SPI 的高级配置参数（SPI 是否使用 DMA 发送数据、SPI 是否使用 DMA 接收数据、QSPI 参数等）。
- uapi_spi_set_extra_attr：设置 SPI 的高级配置参数。
- uapi_spi_select_slave：片选。
- uapi_spi_master_write：SPI 主机半双工发送数据。
- uapi_spi_master_read：SPI 主机半双工接收数据。
- uapi_spi_master_writeread：SPI 主机全双工收发数据。
- uapi_spi_slave_read：SPI 从机半双工接收数据。
- uapi_spi_slave_write：SPI 从机半双工发送数据。
- uapi_spi_set_dma_mode：SPI 设置为手动 DMA 模式。
- uapi_spi_set_irq_mode：SPI 设置为手动中断模式。

2.3 开发指引

SPI 用于对接支持 SPI 协议的设备，SPI 单元可以作为主设备或从设备。以 SPI 单元作为主设备为例，写数据操作如下：

步骤 1 通过 IO 复用，复用 SPI 功能用到的管脚为 SPI 功能。

管脚复用各个功能请参考 platform_core.h 中各个管脚功能的定义。

步骤 2 调用 uapi_spi_init, 初始化 SPI 资源, 选择 SPI 功能单元以及配置 SPI 参数。

```
errcode_t usr_spi_init()
{
    errcode_t ret;
    spi_bus_t bus_id = SPI_BUS_0;
    spi_attr_t attr;
    spi_extra_attr_t extra_attr;

    /* IO 复用 */

    uapi_pin_set_mode(L_MGPIO41, PIN_MODE_2);
    uapi_pin_set_mode(L_MGPIO42, PIN_MODE_2);
    uapi_pin_set_mode(L_MGPIO43, PIN_MODE_2);
    uapi_pin_set_mode(L_MGPIO44, PIN_MODE_2);
    uapi_pin_set_mode(L_MGPIO45, PIN_MODE_2);
    uapi_pin_set_mode(L_MGPIO47, PIN_MODE_2);
    uapi_pin_set_mode(L_MGPIO49, PIN_MODE_2);
    uapi_pin_set_mode(L_MGPIO50, PIN_MODE_2);
    uapi_pin_set_mode(L_MGPIO51, PIN_MODE_2);
    uapi_pin_set_mode(L_MGPIO52, PIN_MODE_2);
    uapi_pin_set_mode(L_MGPIO53, PIN_MODE_2);

    /* 主模式参数设置 */
    spi_attr_t attr = {
        .is_slave = false,
        .salve_num = 1,
        .bus_clk = 32000000,
        .freq_mhz = 2, /* Min freq in standard spi is 1, max is 48 */
        .clk_polarity = (uint32_t)SPI_CFG_CLK_CPOL_0,
        .clk_phase = (uint32_t)SPI_CFG_CLK_CPHA_0,
        .frame_format = SPI_CFG_FRAME_FORMAT_MOTOROLA_SPI,
        .spi_frame_format = (uint32_t)HAL_SPI_FRAME_FORMAT_STANDARD,
        .frame_size = (uint32_t)HAL_SPI_FRAME_SIZE_8,
        .tmod = HAL_SPI_TRANS_MODE_TX,
    };
    spi_extra_attr_t extra_attr = {
        .tx_use_dma = false,
        .rx_use_dma = false
    };
    ret = uapi_spi_init(bus_id, &attr, &extra_attr);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }
    return ERRCODE_SUCC;
}
```

}

步骤 3 调用 `uapi_spi_master_write`，进行 SPI 主设备写操作，初始化时的 `tmod` 参数建议配置为 `HAL_SPI_TRANS_MODE_TX`。

以主设备发送数据为例：

```
errcode_t demo_spi_master_write()
{
    errcode_t ret;

    uint32_t timeout = 100; // 当前传输的超时时间

    uint8_t data[100] = { 0 };
    spi_xfer_data_t xfer_data = {
        .tx_buff = data,
        .tx_bytes = sizeof(data)
    };

    /* 主模式参数设置 */
    spi_attr_t attr = {
        .is_slave = false,
        .salve_num = 1,
        .bus_clk = 32000000,
        .freq_mhz = 2, /* Min freq in standard spi is 1, max is 48 */
        .clk_polarity = (uint32_t)SPI_CFG_CLK_CPOL_0,
        .clk_phase = (uint32_t)SPI_CFG_CLK_CPHA_0,
        .frame_format = SPI_CFG_FRAME_FORMAT_MOTOROLA_SPI,
        .spi_frame_format = (uint32_t)HAL_SPI_FRAME_FORMAT_STANDARD,
        .frame_size = (uint32_t)HAL_SPI_FRAME_SIZE_8,
        .tmod = HAL_SPI_TRANS_MODE_TX,
    };

    uapi_spi_set_attr(SPI_BUS_0, &attr)
    ret = uapi_spi_master_write(SPI_BUS_0, &xfer_data, timeout);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }
    return ERRCODE_SUCC;
}
```

步骤 4 调用 `uapi_spi_master_read`，进行 SPI 主设备读操作，初始化时的 `tmod` 参数建议配置为 `HAL_SPI_TRANS_MODE_RX`。

以主设备读数据为例：

```
errcode_t demo_spi_master_read()
{
```

```

errcode_t ret;
uint32_t timeout = 100; // 当前传输的超时时间

uint8_t data[100] = { 0 };
spi_xfer_data_t xfer_data = {
    .rx_buff = data,
    .rx_bytes = sizeof(data)
};

/* 主模式参数设置 */
spi_attr_t attr = {
    .is_slave = false,
    .salve_num = 1,
    .bus_clk = 32000000,
    .freq_mhz = 2, /* Min freq in standard spi is 1, max is 48 */
    .clk_polarity = (uint32_t)SPI_CFG_CLK_CPOL_0,
    .clk_phase = (uint32_t)SPI_CFG_CLK_CPHA_0,
    .frame_format = SPI_CFG_FRAME_FORMAT_MOTOROLA_SPI,
    .spi_frame_format = (uint32_t)HAL_SPI_FRAME_FORMAT_STANDARD,
    .frame_size = (uint32_t)HAL_SPI_FRAME_SIZE_8,
    .tmod = HAL_SPI_TRANS_MODE_RX,
};
uapi_spi_set_attr(SPI_BUS_0, &attr)
ret = uapi_spi_master_read(SPI_BUS_0, &xfer_data, timeout);
if (ret != ERRCODE_SUCC) {
    return ret;
}
return ERRCODE_SUCC;
}

```

步骤 5 调用 uapi_spi_master_writeread，进行 SPI 主设备先写后读操作，初始化时的 tmod 参数建议配置为 HAL_SPI_TRANS_MODE_TXRX。

```

errcode_t demo_spi_master_writeread()
{
    errcode_t ret;
    uint32_t timeout = 100; // 当前传输的超时时间

    uint8_t r_data[100] = { 0 };
    uint8_t w_data[100] = { 0 };
    spi_xfer_data_t xfer_data = {
        .tx_buff = w_data,
        .tx_bytes = sizeof(w_data),
        .rx_buff = r_data,
        .rx_bytes = sizeof(r_data)
    }
}

```

```

};
/* 主模式参数设置 */
spi_attr_t attr = {
    .is_slave = false,
    .salve_num = 1,
    .bus_clk = 32000000,
    .freq_mhz = 2, /* Min freq in standard spi is 1, max is 48 */
    .clk_polarity = (uint32_t)SPI_CFG_CLK_CPOL_0,
    .clk_phase = (uint32_t)SPI_CFG_CLK_CPHA_0,
    .frame_format = SPI_CFG_FRAME_FORMAT_MOTOROLA_SPI,
    .spi_frame_format = (uint32_t)HAL_SPI_FRAME_FORMAT_STANDARD,
    .frame_size = (uint32_t)HAL_SPI_FRAME_SIZE_8,
    .tmod = HAL_SPI_TRANS_MODE_TXRX,
};

uapi_spi_set_attr(SPI_BUS_0, &attr);

ret = uapi_spi_master_writeread(SPI_BUS_0, &xfer_data, timeout);
if (ret != ERRCODE_SUCC) {
    return ret;
}
return ERRCODE_SUCC;
}

```

步骤 6 调用 uapi_spi_select_slave，进行从设备选择操作。

```

errcode_t demo_spi_slave_select()
{
    errcode_t ret;
    spi_slave_t slave = SPI_SLAVE0;
    ret = uapi_spi_select_slave(SPI_BUS_0, slave);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }
    return ERRCODE_SUCC;
}

```

----结束

SPI 手动 DMA 模式配置方式，SPI 初始化后，配置传输模式，再进行读写操作。

步骤 1 配置手动模式宏定义，请参考“config.py”文件。

步骤 2 调用 uapi_spi_set_dma_mode 接口，配置是否使用手动 DMA 模式。

```
errcode_t demo_spi_set_dma_mode(spi_bus_t bus, bool en)
```

```
{
    errcode_t ret;
    spi_dma_config_t dma_cfg = {
        .src_width = demo_width,
        .dest_width = demo_width,
        .burst_length = demo_burst_length,
        .priority = demo_priority
    };
    ret = uapi_spi_set_dma_mode(bus, en, &dma_cfg);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }
    return ERRCODE_SUCC;
}
```

----结束

SPI 手动中断模式配置方式，SPI 初始化后，配置传输模式，再进行读写操作。

步骤 1 配置手动模式宏定义，请参考“config.py”文件。

步骤 2 调用 uapi_spi_set_irq_mode 接口，配置是否使用手动中断模式。

```
errcode_t demo_spi_set_int_mode(spi_bus_t bus, bool en)
{
    errcode_t ret;
    ret = uapi_spi_set_irq_mode(bus, en, spi_rx_callback, test_spi_write_int_handler);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }
    return ERRCODE_SUCC;
}
```

----结束

2.4 注意事项

- 当不再使用 SPI 时，必须调用 uapi_spi_deinit 进行资源释放，否则再进行初始化时将返回错误。
- 使用 microwire 帧协议时，由于 microwire 帧协议限制，主设备只能发送 8bit 位宽数据。

- SPI 使用约束，做从机时会独占总线，不适用于一主多从做为从机的场景。
- 手动中断模式和自动切换模式互斥。

Draft

3 I2C

- 3.1 概述
- 3.2 功能描述
- 3.3 开发指引
- 3.4 注意事项

3.1 概述

I2C 模块是 APB 总线上的设备，是 I2C 总线上的主设备，支持多主设备时的总线仲裁。该模块内部包含 8bit×8 发送和接收 FIFO 各一个，用于主从设备之间的通信和数据传输。

3.2 功能描述

说明

I2C 支持手动模式和自动模式。

- 手动模式：支持轮询模式读写、中断模式读写以及 DMA 模式读写。

CONFIG_I2C_SUPPORT_DMA：支持 DMA。

CONFIG_I2C_SUPPORT_INT：支持中断。

- 自动模式：支持 DMA 模式和轮询模式自动切换。I2C 自动模式配置宏定义后，按照轮询的读写方式即可。

CONFIG_I2C_SUPPORT_DMA：支持 DMA。

CONFIG_I2C_SUPPORT_POLL_AND_DMA_AUTO_SWITCH：支持自动识别传输模式。

CONFIG_I2C_POLL_AND_DMA_AUTO_SWITCH_THRESHOLD=0x8: 自动识别传输阈值, 大于阈值使用 DMA 模式, 小于等于阈值使用轮询模式, 阈值大小可调。

I2C 模块提供的接口:

- uapi_i2c_master_init: 初始化该 I2C 设备为主机, 需要传入的参数有: 总线号、波特率、高速模式主机码 (只有高速模式需要配置, 范围 0~7)。
- uapi_i2c_slave_init: 初始化该 I2C 设备为从机, 需要传入的参数有: 总线号、波特率、从机地址 (支持 7 地址寻址地址、10 地址寻址地址)。
- uapi_i2c_deinit: 去初始化 I2C 设备, 支持主从机。
- uapi_i2c_master_write: I2C 主机将数据发送到目标从机上, 使用轮询模式。
- uapi_i2c_master_read: 主机接收来自目标 I2C 从机的数据, 使用轮询模式。
- uapi_i2c_master_writeread: 主机发送数据到目标 I2C 从机, 并接收来自此从机的数据, 使用轮询模式。
- uapi_i2c_slave_write: 从机将数据发送给主机, 使用轮询模式。
- uapi_i2c_slave_read: 从机接收来自主机的数据, 使用轮询模式。
- uapi_i2c_set_baudrate: 对已初始化的 I2C 重置波特率, 支持主从机。
- uapi_i2c_set_dma_mode: 设置 DMA 读写模式。
- uapi_i2c_set_irq_mode: 设置中断读写模式。

3.3 开发指引

I2C 用于对接支持 I2C 协议的设备, I2C 单元可以作为主设备或从设备。以 I2C 单元作为主设备为例:

步骤 1 通过 IO 复用, 将用到的管脚复用为 I2C 功能。

步骤 2 调用 uapi_i2c_init 接口, 初始化 I2C 资源, 此处以初始化 I2C 主机为例:

```
errcode_t sample_i2c_init(uint32_t bus_id, uint32_t baudrate, uint8_t hscode)
{
    errcode_t ret;
    /* IO 复用 */

    uapi_pin_set_mode(I2C_1_SCL_PIN, PIN_MODE_1);
    uapi_pin_set_mode(I2C_1_SDA_PIN, PIN_MODE_1);
    uapi_pin_set_mode(I2C_2_SCL_PIN, PIN_MODE_1);
    uapi_pin_set_mode(I2C_2_SDA_PIN, PIN_MODE_1);
    ret = uapi_i2c_master_init(bus_id, baudrate, hscode);
}
```

```

    if (ret != ERRCODE_SUCC) {
        return ret;
    }
    return ERRCODE_SUCC;
}

```

步骤 3 调用 uapi_i2c_master_write 接口，实现主机发送数据。

```

errcode_t sample_i2c_write(uint32_t bus_id, uint16_t dev_addr, uint8_t *send_buf, uint32_t
send_len)
{
    errcode_t ret;
    i2c_data_t data = { 0 };
    data.send_buf = send_buf;
    data.send_len = send_len;
    ret = uapi_i2c_master_write(bus_id, dev_addr, &data);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }
    return ERRCODE_SUCC;
}

```

步骤 4 调用 uapi_i2c_master_read 接口，实现主机读数据。

```

errcode_t sample_i2c_read(uint32_t bus_id, uint16_t dev_addr, uint8_t *recv_buf, uint32_t recv_len)
{
    errcode_t ret;
    i2c_data_t data = { 0 };
    data.receive_buf = recv_buf;
    data.receive_len = recv_len;
    ret = uapi_i2c_master_read(bus_id, dev_addr, &data);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }
    return ERRCODE_SUCC;
}

```

----结束

I2C 手动 DMA 模式配置方式，I2C 初始化后，配置传输模式，再进行读写操作。

步骤 1 配置手动模式宏定义，请参考“config.py”文件。

步骤 2 调用 uapi_i2c_set_dma_mode 接口，配置是否使用手动 DMA 模式。

```

errcode_t demo_i2c_set_dma_mode(i2c_bus_t bus, bool en)

```

```
{
    errcode_t ret;
    ret = uapi_i2c_set_dma_mode(bus, en);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }
    return ERRCODE_SUCC;
}
```

----结束

I2C 手动中断模式配置方式，I2C 初始化后，配置传输模式，再进行读写操作。

步骤 1 配置手动模式宏定义，请参考“config.py”文件

步骤 2 调用 uapi_i2c_set_irq_mode 接口，配置是否使用手动中断模式。

```
errcode_t demo_i2c_set_int_mode(i2c_bus_t bus, bool en)
{
    errcode_t ret;
    ret = uapi_i2c_set_irq_mode(bus, en);
    if (ret != ERRCODE_SUCC) {
        return ret;
    }
    return ERRCODE_SUCC;
}
```

----结束

3.4 注意事项

- 函数 uapi_i2c_set_baudrate 需要先初始化，再调用，方便修改波特率。如果 I2C 未经过初始化，直接调用 uapi_i2c_set_baudrate 函数，会返回错误码 ERRCODE_I2C_NOT_INIT。
- 需要主动确保数据发送指针 send_buf 和数据接收指针 receive_buf 不可传入空指针。
- 当发送的数据大于对接设备的可接受范围时，会发送失败；如果发送数据失败，再切换另一个 I2C 设备继续发送时，会造成总线挂死，所有 I2C 设备都无法正确发送数据。

- uapi_i2c_master_init 不能多次初始化，使用完成后需要调用 uapi_i2c_deinit 进行去初始化。
- 手动中断模式和自动切换模式互斥。

Draft

4 ADC

- [4.1 概述](#)
- [4.2 功能描述](#)
- [4.3 开发指引](#)
- [4.4 注意事项](#)

4.1 概述

ADC (Analog-to-Digital Converter) 实现对外部模拟信号转换成一定比例的数字值, 从而实现对模拟信号的测量, 可应用于电量检测、按键检测等。

4.2 功能描述

ADC 模块提供的接口:

- uapi_adc_init: 初始化 ADC。
- uapi_adc_deinit: 去初始化 ADC。
- uapi_adc_power_en: 上下电并启用或关闭 ADC。
- uapi_adc_is_using: 检查 ADC 是否正在使用。
- uapi_adc_open_channel: 开启一个 ADC 通道。
- uapi_adc_close_channel: 关闭一个 ADC 通道。
- uapi_adc_manual_sample: ADC 手动采样。
- uapi_adc_auto_scan_ch_enable: ADC 开始自动采样接口。

- uapi_adc_auto_scan_ch_disable: ADC 停止自动采样接口。

4.3 开发指引

4.3.1 ADC 手动采样开发指引

芯片提供 1 个 ADC、7 个独立通道。可以选择采样次数、丢弃次数（可选）来调用 uapi_adc_manul_sample 接口获取采样值，可通过转换获得电压。

示例：

```
/* 采样次数 */
#define TESTADC_SAMPLE_NUMS 10

/* 采样值转换为电压 */

#define adc_trans_sytick_to_voltage_without_auto_scan(x) (((float)((float)(x) * 18) / 40960))

/* 采样数转为电压值 */
static float test_adc_stick_transfer_voltage(uint16_t stick)
{
    return adc_trans_sytick_to_voltage_without_auto_scan(stick);
}

/* 手动 */
#define ADC_SAMPLE_TIMES 10
#define ADC_SAMPLE_DATA_LEN 10010
uint32_t g_adc_sample_times = 0;
uint32_t g_save_adc_index = 0;
uint32_t *g_save_adc_data = NULL;
static uint32_t adc_auto_sample_show_data(uint8_t *para, uint32_t para_len)
{
    wstp_print("g_save_adc_index = %d\r\n", g_save_adc_index);
    for (uint32_t i = 0; i < g_save_adc_index; i++) {
        wstp_print("%d\r\n", g_save_adc_data[i]);
    }

    g_save_adc_index = 0;
    g_adc_sample_times = 0;
    free(g_save_adc_data);
    g_save_adc_data = NULL;
}
```

```

        return ERRCODE_SUCC;
    }

static void adc_autosample_callback(uint8_t channel, uint32_t *buffer, uint32_t length, bool *next)
{
    unused(next);
    if (g_adc_sample_times >= ADC_SAMPLE_TIMES) {
        wstp_print("g_adc_sample_times = %d\r\n", g_adc_sample_times);
        uapi_adc_auto_scan_ch_disable(channel);
        return;
    }

    memcpy_s(g_save_adc_data + g_save_adc_index, length * sizeof(uint32_t), buffer, length *
sizeof(uint32_t));
    g_save_adc_index += length;
    g_adc_sample_times++;
    wstp_print("length = %d\r\n", length);
    return;
}

static uint32_t adc_auto_sample_by_diag(uint8_t *para, uint32_t para_len)
{
    char *ptr = (char *)para;
    char *token = NULL;
    char *next_token = NULL;
    uint32_t channel = 0;
    uint32_t freq = 0;
    uint32_t sample_time = 0;
    uint32_t idx = 0;
    uint32_t tmp[ADC_AUTODATA_PRT_INDEX3] = {0};
    token = strtok_s((char*)ptr, ",", &next_token);
    while (token != NULL) {
        tmp[idx++] = strtoul(token, NULL, 10); // 10: 十进制

        token = strtok_s(NULL, ",", &next_token);
    }
    channel = tmp[0];
    freq = tmp[ADC_AUTODATA_PRT_INDEX1];
    sample_time = tmp[ADC_AUTODATA_PRT_INDEX2];

    adc_scan_config_t config = {0};
    config.freq = (uint8_t)freq;
    config.long_sample_time = sample_time;

```

```

g_save_adc_data = (uint32_t *)malloc(sizeof(uint32_t) * ADC_SAMPLE_DATA_LEN);
if (g_save_adc_data == NULL) {
    wstp_print("malloc g_save_adc_data fail!\n\n");
    return ERRCODE_FAIL;
}
(void)uapi_adc_init(ADC_CLOCK_500KHZ);
(void)uapi_adc_power_en(AFE_SCAN_MODE_MAX_NUM, true);
return(uapi_adc_auto_scan_ch_enable((uint8_t)channel, config, adc_autosample_callback));
}

static uint32_t adc_auto_sample_disable(uint8_t *para, uint32_t para_len)
{
    char *ptr = (char *)para;
    uint32_t ret = 0;

    uint32_t channel = strtoul(ptr, NULL, 10); // 10: 十进制

    wstp_print("disable adc auto sample , channel = %x\n\n", channel);

    return(uapi_adc_auto_scan_ch_disable(channel));
}

/* 采样 */
static bool test_adc_get_sample_result(float *voltage, uint8_t channel)
{
    unsigned int sample_number = TESTADC_SAMPLE_NUMS;
    unsigned int stick = 0;
    unsigned int sample_zero_count = 0;
    /* 遍历采样 */
    for (unsigned int i = 0; i < sample_number; i++) {
        osal_irq_lock();
        unsigned int sample_result = uapi_adc_manual_sample();
        if (sample_result == 0) {
            sample_zero_count++;
        }
        stick += sample_result;
        osal_irq_unlock();
    }
    sample_number -= sample_zero_count;
    if (sample_number > 0) {
        stick = (stick / sample_number);
    } else {
        *voltage = 0;
        return false;
    }
}

```



```
/* 采样数转为电压值 */  
  
*voltage = test_adc_stick_transfer_voltage((uint16_t)stick);  
return true;  
}
```

4.3.2 ADC 自动采样开发指引

```
#define ADC_SAMPLE_TIMES 10  
#define ADC_SAMPLE_DATA_LEN 10010  
uint32_t g_adc_sample_times = 0;  
uint32_t g_save_adc_index = 0;  
uint32_t *g_save_adc_data = NULL;  
static uint32_t adc_auto_sample_show_data(uint8_t *para, uint32_t para_len)  
{  
    wstp_print("g_save_adc_index = %d\r\n", g_save_adc_index);  
    for (uint32_t i = 0; i < g_save_adc_index; i++) {  
        wstp_print("%d\r\n", g_save_adc_data[i]);  
    }  
  
    g_save_adc_index = 0;  
    g_adc_sample_times = 0;  
    free(g_save_adc_data);  
    g_save_adc_data = NULL;  
    return ERRCODE_SUCC;  
}  
/* 自动采样ADC数据回调函数 */  
static void adc_autosample_callback(uint8_t channel, uint32_t *buffer, uint32_t length, bool *next)  
{  
    unused(next);  
    if (g_adc_sample_times >= ADC_SAMPLE_TIMES) {  
        wstp_print("g_adc_sample_times = %d\r\n", g_adc_sample_times);  
        uapi_adc_auto_scan_ch_disable(channel);  
        return;  
    }  
  
    memcpy_s(g_save_adc_data + g_save_adc_index, length * sizeof(uint32_t), buffer, length *  
sizeof(uint32_t));  
    g_save_adc_index += length;  
    g_adc_sample_times++;  
    wstp_print("length = %d\r\n", length);  
    return;  
}
```

```

static uint32_t adc_auto_sample_by_diag(uint8_t *para, uint32_t para_len)
{
    char *ptr = (char *)para;
    char *token = NULL;
    char *next_token = NULL;
    uint32_t channel = 0;
    uint32_t freq = 0;
    uint32_t sample_time = 0;
    uint32_t idx = 0;
    uint32_t tmp[ADC_AUTODATA_PRT_INDEX3] = {0};
    token = strtok_s((char*)ptr, ",", &next_token);
    while (token != NULL) {
        tmp[idx++] = strtoul(token, NULL, 10); // 10: 十进制
        token = strtok_s(NULL, ",", &next_token);
    }
    channel = tmp[0];
    freq = tmp[ADC_AUTODATA_PRT_INDEX1];
    sample_time = tmp[ADC_AUTODATA_PRT_INDEX2];

    adc_scan_config_t config = {0};
    config.freq = (uint8_t)freq;
    config.long_sample_time = sample_time;

    g_save_adc_data = (uint32_t *)malloc(sizeof(uint32_t) * ADC_SAMPLE_DATA_LEN);
    if (g_save_adc_data == NULL) {
        wstp_print("malloc g_save_adc_data fail!\n\n");
        return ERRCODE_FAIL;
    }
    (void)uapi_adc_init(ADC_CLOCK_500KHZ);
    (void)uapi_adc_power_en(AFE_SCAN_MODE_MAX_NUM, true);
    return(uapi_adc_auto_scan_ch_enable((uint8_t)channel, config, adc_autosample_callback));
}

static uint32_t adc_auto_sample_disable(uint8_t *para, uint32_t para_len)
{
    char *ptr = (char *)para;
    uint32_t ret = 0;
    uint32_t channel = strtoul(ptr, NULL, 10); // 10: 十进制

    wstp_print("disable adc auto sample , channel = %x\n\n", channel);

    return(uapi_adc_auto_scan_ch_disable(channel));
}

```

```
}
```

4.4 注意事项

ADC 自动采样与手动采样互斥。自动采样需要开启宏：

'CONFIG_ADC_SUPPORT_AUTO_SCAN'、

'CONFIG_ADC_SUPPORT_LONG_SAMPLE'，手动采样需要关闭这两个宏。具体设置方法请参见《Sparta 软件开发指南》。

ADC 自动采样支持的采样频率有 100Hz、200Hz、250Hz、500Hz、1000Hz。

5 Pinctrl/GPIO

5.1 概述

5.2 功能描述

5.3 开发指引

5.4 注意事项

5.1 概述

Pinctrl 驱动支持配置 IO 驱动能力、复用 IO 为外设管脚、设置上下拉状态等功能。

GPIO 驱动支持设置 GPIO 管脚方向、设置输出电平状态、中断上报等功能。

5.2 功能描述

Pinctrl 模块提供的接口：

- uapi_pin_init：初始化 Pinctrl。
- uapi_pin_deinit：去初始化 Pinctrl。
- uapi_pin_set_mode：设置指定 IO 复用模式。
- uapi_pin_get_mode：获取指定 IO 的复用模式。
- uapi_pin_set_ds：设置指定 IO 驱动能力。
- uapi_pin_get_ds：获取指定 IO 驱动能力。
- uapi_pin_set_pull：设置指定 IO 的上拉/下拉功能。
- uapi_pin_get_pull：获取指定 IO 的上拉/下拉状态。

- uapi_pin_set_ie: 设置 pin 脚的输入使能。
- uapi_pin_get_ie: 获取 pin 脚的输入使能状态。

GPIO 模块提供的接口:

- uapi_gpio_init: 初始化 GPIO。
- uapi_gpio_deinit: 去初始化 GPIO。
- uapi_gpio_set_dir: 设置指定 GPIO 方向 (输入/输出)。
- uapi_gpio_set_val: 设置指定 GPIO 电平状态。
- uapi_gpio_get_val: 获取指定 GPIO 电平状态。
- uapi_gpio_register_isr_func: 注册指定 GPIO 中断。
- uapi_gpio_unregister_isr_func: 去注册指定 GPIO 中断。
- uapi_gpio_disable_interrupt: 关闭 GPIO 中断。
- uapi_gpio_enable_interrupt: 使能 GPIO 中断。
- uapi_gpio_clear_interrupt: 清除 GPIO 中断。
- uapi_gpio_toggle: GPIO 输出电平状态翻转。

5.3 开发指引

Pinctrl 接口使用遵循如下操作步骤 (以下步骤根据实际需要可选):

步骤 1 调用 uapi_pin_set_mode、uapi_pin_get_mode 接口, 设置/查看指定 IO 的复用模式。

步骤 2 调用 uapi_pin_set_ds、uapi_pin_get_ds 接口, 设置/查看指定 IO 的驱动能力。

步骤 3 调用 uapi_pin_set_pull、uapi_pin_get_pull 接口, 设置/查看指定 IO 的上拉/下拉状态。

----结束

Pinctrl 接口和 GPIO 接口独立使用, GPIO 接口使用遵循如下操作步骤:

步骤 1 调用 uapi_pin_set_mode 接口, 将 PIN 复用为 GPIO 功能。

步骤 2 根据用户开发需求, 可设置 GPIO 接口为输出, 输入和中断模式。

- **输出:**
 - a. 调用 uapi_gpio_set_dir 接口, 设置 GPIO 方向为 OUT。
 - b. 调用 uapi_gpio_set_val 接口, 设置 GPIO 输出电平状态 (高/低)。

- **输入:**
 - a. 调用 uapi_gpio_set_dir 接口, 设置 GPIO 方向为 IN。
 - b. 调用 uapi_gpio_get_val 接口, 获取 GPIO 输入电平状态。
- **中断:**
 - a. 调用 uapi_gpio_set_dir 接口, 设置 GPIO 方向为 IN。
 - b. 调用 uapi_gpio_register_isr_func 接口, 注册 GPIO 中断回调函数。
 - c. 调用 uapi_gpio_unregister_isr_func 接口, 注销 GPIO 中断回调函数 (去注册中断时调用)。

----结束

示例:

```
void gpio_callback_func(pin_t pin, uintptr_t param)
{
    unused(param);
    printf("PIN:%d interrupt success. \r\n", pin);
}

errcode_t test_gpio_isr(pin_t pin)
{
    uapi_pin_init();
    uapi_gpio_init();

    uapi_pin_set_mode(pin, HAL_PIO_FUNC_GPIO); /* 设置指定IO复用模式 */

    uapi_gpio_set_dir(pin, GPIO_DIRECTION_INPUT); /* 设置指定GPIO为输入模式*/

    /* 注册指定GPIO上升沿中断, 回调函数为gpio_callback_func */

    if (uapi_gpio_register_isr_func(pin, GPIO_INTERRUPT_RISING_EDGE, gpio_callback_func))
    {
        uapi_gpio_unregister_isr_func(pin); /* 清理残留 */

        return ERRCODE_FAIL;
    }

    return ERRCODE_SUCCESS;
}
```

5.4 注意事项

- 配置 IO 复用功能时，应关注此 IO 是否已经被复用为其他功能，避免影响既有功能，IO 复用请参考发布 SDK 中 platform_core.h 中的定义。SFC 相关的 IO 复用和配置，仅供 SDK 内部使用，切勿复用为其它功能，否则将导致系统异常。
- 在使用 GPIO 电平中断时，需要在回调函数中添加中断处理或控制输入电平触发中断时间，否则会导致系统一直处于中断处理中，无法执行其他功能。
- 触发方式 (trigger) 在没有明确需求场景时，推荐使用默认配置。

6 DMA

- 6.1 概述
- 6.2 功能描述
- 6.3 开发指引
- 6.4 注意事项

6.1 概述

直接存储器访问 DMA (Direct Memory Access) 方式是一种完全由硬件执行 I/O 交换的工作方式。在这种方式中, 直接存储器访问控制器 DMAC (Direct Memory Access Controller) 直接在存储器和外设、外设和外设、存储器和存储器之间进行数据传输, 减少处理器的干涉和开销。

DMA 方式一般用于高速传输成组的数据。DMAC 在收到 DMA 传输请求后根据 CPU 对通道的配置启动总线主控制器, 向存储器和外设发出地址和控制信号, 对传输数据的个数计数, 并且以中断方式向 CPU 报告传输操作的结束或错误。

6.2 功能描述

📖 说明

如果需要在 SPI/UART/I2C 中使用 DMA 传输数据, 需要在系统启动时进行 DMA 初始化。

DMA 模块提供的接口:

- uapi_dma_init: 初始化 DMA。
- uapi_dma_deinit: 去初始化 DMA。

- uapi_dma_open: 打开 DMA。
- uapi_dma_close: 关闭 DMA。
- uapi_dma_start_transfer: 启动指定通道的 DMA 传输。
- uapi_dma_end_transfer: 停止指定通道的 DMA 传输。
- uapi_dma_transfer_memory_single: 通过 DMA 通道传输类型为内存到内存的数据。
- uapi_dma_configure_peripheral_transfer_single: 通过 DMA 通道传输类型为内存到外设或外设到内存的数据。
- uapi_dma_enable_lli: 启用 DMA 链表传输。
- uapi_dma_transfer_memory_lli: 通过 DMA 通道以链表模式传输类型为内存到内存的数据。
- uapi_dma_configure_peripheral_transfer_lli: 通过 DMA 通道以链表模式传输类型为内存到外设或外设到内存的数据。

6.3 开发指引

DMA 接口仅对外提供存储器到存储器的拷贝功能（其他拷贝方式可参考外设驱动），操作步骤如下：.

步骤 1 调用 uapi_dma_init 接口，初始化 DMA 模块。

步骤 2 调用 uapi_dma_open_ch 接口，打开 DMA 通道。

步骤 3 调用 uapi_dma_transfer 接口，DMA 开始传输，通过参数 block 可以设置是否为阻塞模式。

----结束

示例：

```
#define TEST_DMA_TRF_WORD_NUM 500
static unsigned int g_dma_src_data[TEST_DMA_TRF_WORD_NUM];
static unsigned int g_dma_desc_data[TEST_DMA_TRF_WORD_NUM];

/* 传输完成后回调函数处理 */
static void test_dma_trans_done_callback(hal_dma_interrupt_t int_type)
{
    switch (int_type) {
```

```

        case HAL_DMA_INTERRUPT_TFR:
            g_dma_trans_done = true;
            g_dma_trans_succ = true;
            break;
        case HAL_DMA_INTERRUPT_BLOCK:
            g_dma_trans_done = true;
            g_dma_trans_succ = true;
            break;
        case HAL_DMA_INTERRUPT_ERR:
            g_dma_trans_done = true;
            g_dma_trans_succ = false;
            break;
        default:
            break;
    }
    printf("[DMA] int_type is %d. \r\n", int_type);
}

static void test_fill_test_buffer(void *data, unsigned int length)
{
    for (unsigned int i = 0; i < length; i++) {
        *((unsigned char *)data + i) = (unsigned char)i;
    }
}

static void test_clear_test_buffer(void *data, unsigned int length)
{
    memset_s(data, length, 0, length);
}

errcode_t test_dma_mem_to_mem_single()
{
    dma_ch_user_memory_config_t transfer_config;

    /* 填充源地址要发送的数据 */
    test_fill_test_buffer((void*)(uintptr_t)g_dma_src_data, sizeof(g_dma_src_data));

    /* 清空目的地址的数据 */
    test_clear_test_buffer((void*)(uintptr_t)g_dma_desc_data, sizeof(g_dma_desc_data));

    /* 初始化dma */
    uapi_dma_init();

    /* 开启DMA模块 */
    uapi_dma_open();

    /* 源地址 */
    transfer_config.src = ((uint32_t)(uintptr_t)g_dma_src_data);

```

```
/* 目的地址 */
transfer_config.dest = ((uint32_t)(uintptr_t)g_dma_desc_data);
/* 传输数目 */
transfer_config.transfer_num = 100;
/* 优先级 0-3, 0最低 */
transfer_config.priority = 0;
/* 传输宽度 0:1字节 1:2字节 2:4字节 */
transfer_config.width = 0;
/* 调用接口按块发送函数，并注册回调函数 */
if (uapi_dma_transfer_memory_single(&transfer_config, test_dma_trans_done_callback) !=
    ERRCODE_SUCC) {
    return ERRCODE_FAIL;
}
/* 等待发送完成 */
while (!g_dma_trans_done) {}
if (!g_dma_trans_succ) {
    return ERRCODE_FAIL;
}
return ERRCODE_SUCC;
}
```

6.4 注意事项

- 建议仅在需要非阻塞进行数据拷贝的场景下使用 DMA，这样可让出 CPU，传输完成之后 CPU 会上报中断，可以在回调函数中根据事件类型判断传输成功与失败。传输阻塞场景下，仍建议使用 memcpy_s 进行数据拷贝。
- 由于地址映射限制，通用外设（I2C、UART 等）使用 DMA 搬运时，如果数据源涉及内存申请，请使用 osal_malloc 接口申请。

7 PWM

7.1 概述

7.2 功能描述

7.3 开发指引

7.4 注意事项

7.1 概述

PWM (Pulse Width Modulation) 模块通过对一系列脉冲的宽度进行调制, 等效出所需要的波形。即对模拟信号电平进行数字编码, 通过调节频率、占空比的变化来调节信号的变化。主要用于输出波形, 共有 6 个端口, 调用 PWM 端口时需要进行 IO 复用。

7.2 功能描述

PWM 模块提供的接口:

- uapi_pwm_init: 初始化 PWM。
- uapi_pwm_deinit: 去初始化 PWM。
- uapi_pwm_open: 打开 PWM 通道。
- uapi_pwm_close: 关闭 PWM 通道。
- uapi_pwm_register_interrupt: 为 PWM 注册中断回调。
- uapi_pwm_unregister_interrupt: 去注册中断回调。
- uapi_pwm_start: 启动 PWM 信号输出。

- uapi_pwm_stop: 停止 PWM 信号输出。

7.3 开发指引

PWM 利用微处理器的数字输出对模拟电路进行控制，操作步骤如下：

步骤 1 将 IO 复用为 PWM 功能。

步骤 2 调用 uapi_pwm_init 对 PWM 进行初始化。

步骤 3 调用 uapi_pwm_open，配置 PWM 参数，打开指定通道。

步骤 4 调用 uapi_pwm_register_interrupt 接口，注册 pwm 中断的回调函数。

步骤 5 调用 uapi_pwm_start 接口，开启指定 id 的 PWM 信号输出。

步骤 6 调用 uapi_pwm_close 接口，停止指定 id 的 PWM 信号输出。

步骤 7 调用 uapi_pwm_deinit 接口，去初始化指定 id 的 PWM。

----结束

示例：

```
#define TEST_MAX_TIMES 10
#define TEST_TCXO_DELAY_MS 1000
/* pwm注册中断回调函数 */
static errcode_t pwm_test_callback(pwm_channel_t channel)
{
    printf("PWM channel number is %d, func of interrupt start. \r\n", channel);
    uapi_pwm_isr(channel);
    return ERRCODE_SUCC;
}
void test_pwm_sample(pin_t pin, pin_mode_t mode, pwm_channel_t channel)
{
    pwm_channel_t channel;

    /* 设置循环次数 */
    unsigned int test_times;

    /* 配置 low_time, high_time, cycles, repeat. 当repeat为true时候, cycles无效 */
    pwm_config_t cfg_repeat = { 100, 100, 0xFF, true };

    /* 设置可作为pwm io的模式 */
    uapi_pin_set_mode(pin, mode);
```

```
uapi_pwm_init();
/* 打开指定channel的pwm */
uapi_pwm_open(channel, &cfg_repeat);
/* 注册回调函数 */
uapi_pwm_register_interrupt(channel, pwm_test_callback);
/* 启动指定channel pwm输出 */
uapi_pwm_start(channel);
/* 当前设置为循环输出，循环TEST_MAX_TIMES次，每次delay TEST_TCXO_DELAY_MS,后
关闭pwm输出 */
for (test_times = 0; test_times <= TEST_MAX_TIMES; test_times++) {
    if (test_times == TEST_MAX_TIMES) {
        uapi_pwm_close(channel);
        printf("now close the pwm output and trigger interrupt \r\n");
    }
    uapi_tcxo_delay_ms(TEST_TCXO_DELAY_MS);
}
uapi_pwm_deinit();
return;
}
```

7.4 注意事项

- 在调用 uapi_pwm_deinit 接口之前，需要先调用 uapi_pwm_close 接口。
- PWM 调用 uapi_pwm_stop/uapi_pwm_close 时不支持在中断中调用。
- PWM 不支持占空比为 0。
- PWM 不支持多路互补输出。
- 深睡唤醒之后需要先配置复用关系，再调用 uapi_pwm_deinit 接口去初始化，之后调用 uapi_pwm_init 接口初始化。

8 Watchdog

8.1 概述

8.2 功能描述

8.3 开发指引

8.4 注意事项

8.1 概述

Watchdog 用于系统异常情况下，一定时间内（产品默认 8s，可配置）未得到及时踢狗（也称喂狗）从而主动发出复位信号以复位整个系统使系统恢复正常状态。

该模块实现对看门狗的使能/关闭、喂狗、设置超时时间及获取看门狗剩余时间等功能。

8.2 功能描述

Watchdog 模块提供的接口：

- uapi_watchdog_init：初始化 Watchdog 功能，设置看门狗超时时间，单位 s。
- uapi_watchdog_deinit：去初始化 Watchdog 功能。
- uapi_watchdog_set_time：设置看门狗超时时间，单位 s；如不设置，默认时间是 8s。
- uapi_watchdog_enable：使能看门狗。
- uapi_watchdog_kick：喂狗，重新启动计数器，即踢狗。
- uapi_watchdog_disable：关闭看门狗。

- uapi_watchdog_get_left_time: 获取看门狗剩余时间, 单位 ms。

8.3 开发指引

Watchdog 一般用于检测是否死机, 如果超过踢狗等待的时间没有进行踢狗操作, 根据 Watchdog 配置的使能模式产生一个系统复位或者上报狗中断。在需要踢狗的任务场景下按照以下步骤操作:

步骤 1 调用 uapi_watchdog_init 初始化并设置看门狗超时时间。

步骤 2 调用 uapi_watchdog_enable, 使能看门狗模块, 可配置复位模式和中断模式。

步骤 3 调用 uapi_watchdog_kick, 进行踢狗操作, 目前在 idle 任务中已实现了踢狗操作。

步骤 4 调用 uapi_watchdog_get_left_time, 获取看门狗定时器剩余时间 (可选)。

步骤 5 调用 uapi_watchdog_disable, 关闭看门狗 (正常情况下不建议关闭看门狗)。

----结束

示例:

```
void test_wdg()
{
    /* 设置开门狗超时时间 */
    uapi_watchdog_init(32);/* 设置超时时间 */
    uapi_watchdog_enable(WDT_MODE_RESET);/* 使能看门狗 */
    uapi_tcxo_delay_ms(5000); /* delay 5000 ms */
    uapi_watchdog_kick(); /* 踢狗 */

    sample_remain_ms = uapi_watchdog_get_left_time(); /* 获取剩余超时时间 */

    uapi_watchdog_disable();/* 关闭看门狗 */
}
```

8.4 注意事项

- 如果获取时间为 0xFFFFFFFF, 说明看门狗处于未使能状态。

- 看门狗在 SDK 中已经使能且已存在喂狗动作，在特殊场景下，如需长时间占用 CPU，可将看门狗去使能或在业务代码中增加喂狗操作，避免正常业务场景引起看门狗复位。
- 看门狗默认超时时间为 8s，二次触发将引起复位。一般情况下，不建议修改超时时间。

Draft

9 Timer

9.1 概述

9.2 功能描述

9.3 开发指引

9.4 注意事项

9.1 概述

Timer 主要实现软件定时器、高精度的硬件定时器（ μs 级）及计数功能，该模块提供了定时器的创建、删除、开启和停止功能。

需要设定小于 $500\mu\text{s}$ 定时器时，请使用高精度硬件定时器。

Timer 是基于外部 32MHz 晶体为系统提供高精度定时器的功能。

9.2 功能描述

Timer 模块提供的接口：

- uapi_timer_adapter：适配定时器配置。
- uapi_timer_init：初始化 Timer。
- uapi_timer_deinit：去初始化 Timer。
- uapi_timer_create：创建定时器。
- uapi_timer_delete：删除指定定时器。
- uapi_timer_start：开启指定定时器，开始计时。

- `uapi_timer_stop`: 停止当前定时器计时。
- `uapi_get_time_us`: 获取当前计时值。
- `uapi_timer_start_high_precision`: 启动高精度定时器。
- `uapi_timer_reset_high_precision`: 重置高精度定时器。
- `uapi_timer_stop_high_precision`: 停止指定高精度定时器。

9.3 开发指引

步骤 1 调用 `uapi_timer_adapter` 接口，配置定时器索引、定时器中断号和中断优先级。

步骤 2 调用 `uapi_timer_init` 接口，初始化定时器功能。

步骤 3 调用 `uapi_timer_create` 接口，创建一个高精度定时器，函数参数句柄为唯一定时器标识。

步骤 4 调用 `uapi_timer_start` 接口，设置超时时间、超时回调函数、回调函数入参以及启动定时器。

步骤 5 调用 `uapi_timer_stop` 接口，停止当前定时器计时。

步骤 6 调用 `uapi_timer_delete` 接口，删除当前定时器。

步骤 7 调用 `uapi_timer_start_high_precision` 接口，设置定时器硬件 `index`、上报类型、超时时间、中断信息、超时回调函数用以启动定时器。

步骤 8 调用 `uapi_timer_reset_high_precision` 接口，设置定时器硬件 `index`、上报类型、超时时间用以重置定时器超时时间。

步骤 9 调用 `uapi_timer_stop_high_precision` 接口，设置定时器硬件 `index` 用以停止定时器。

----结束

9.4 注意事项

- Timer 创建的定时器超时时间单位为 μs 。
- 默认最多可同时创建 8 个高精度定时器。
- 在非低功耗模式下，Timer 可配置的最大超时时间约为 134s。
- 确定不需要使用当前高精度定时器后，需要调用 `uapi_timer_delete` 接口，释放该定时器资源。

- 禁止在 Timer 的回调函数中调用 `uapi_timer_stop`、`uapi_timer_delete` 等接口。
- 支持 4 路 Timer，其中用户可用的通路是 `TIMER_INDEX_0/TIMER_INDEX_1/TIMER_INDEX_2`。
- 高精度定时器的接口，`index` 参数必须传入空闲的硬件 Timer。

Draft

10 RTC

10.1 概述

10.2 功能描述

10.3 开发指引

10.4 注意事项

10.1 概述

RTC 主要实现定时器（ms 级）功能，该模块提供了定时器的创建、删除、开启和停止功能。

RTC 是基于外部 32.768kHz 晶振或内部 32kHz 时钟为系统提供定时器功能。

10.2 功能描述

RTC 模块提供的接口：

- uapi_rtc_adapter：适配定时器配置。
- uapi_rtc_init：初始化 RTC。
- uapi_rtc_deinit：去初始化 RTC。
- uapi_rtc_create：创建定时器。
- uapi_rtc_delete：删除指定定时器。
- uapi_rtc_start：开启指定定时器，开始计时。
- uapi_rtc_stop：停止当前定时器计时。

10.3 开发指引

步骤 1 调用 `uapi_rtc_adapter` 接口，配置定时器索引、定时器中断号和中断优先级。

步骤 2 调用 `uapi_rtc_init` 接口，初始化定时器功能。

步骤 3 调用 `uapi_rtc_create` 接口，创建一个定时器，函数参数句柄为唯一定时器标识。

步骤 4 调用 `uapi_rtc_start` 接口，设置超时时间、超时回调函数、回调函数入参以及启动定时器。

步骤 5 调用 `uapi_rtc_stop` 接口，停止当前定时器计时。

步骤 6 调用 `uapi_rtc_delete` 接口，删除当前定时器。

----结束

10.4 注意事项

- RTC 创建的定时器超时时间单位为 ms。
- 默认最多可同时创建 16 个 RTC 软件定时器。
- RTC 定时器可以在低功耗睡眠后唤醒使用。
- 确定不需要使用当前定时器后，需要调用 `uapi_rtc_delete` 接口，释放该定时器资源。
- 禁止在 RTC 的回调函数中调用 `uapi_rtc_stop`、`uapi_rtc_delete` 等接口。

11 SysTick

11.1 概述

11.2 功能描述

11.3 开发指引

11.4 注意事项

11.1 概述

SYSTEM TICK 是基于外部 32.768kHz 晶振或内部 32kHz 时钟为系统提供计数的功能。

11.2 功能描述

SysTick 模块提供的接口：

- uapi_systick_init：初始化 SysTick。
- uapi_systick_deinit：去初始化 SysTick。
- uapi_systick_count_clear：清除 SysTick 计数。
- uapi_systick_get_count：获取 SysTick 计数值。
- uapi_systick_get_s：获取 SysTick 计数秒值。
- uapi_systick_get_ms：获取 SysTick 计数毫秒值。
- uapi_systick_get_us：获取 SysTick 计数微秒值。
- uapi_systick_delay_count：按 count 计数延时。
- uapi_systick_delay_s：按秒数延时。

- uapi_systick_delay_ms: 按毫秒数延时。
- uapi_systick_delay_us: 按微秒数延时。

11.3 开发指引

步骤 1 调用 uapi_systick_init 接口，初始化 Systick 模块。

步骤 2 调用 uapi_systick_get_count 接口，获取当前 Systick 计数值。

步骤 3 调用 uapi_systick_delay_ms 接口，延迟传入的时间。

----结束

示例:

```
void test_systick_delayms(void)
{
    uint64_t count_before_delay_count;
    uint64_t count_after_delay_count;
    /* systick模块初始化 */
    uapi_systick_init();
    /* 通过count数差值验证延迟时间 */
    count_before_delay_count= uapi_systick_get_count();
    uapi_systick_delay_ms(1000);
    count_after_delay_count = uapi_systick_get_count();
    printf("test case delay count %lu.\r\n",count_before_delay_count - count_after_delay_count);
    return;
}
```

11.4 注意事项

无。

12 TCXO

12.1 概述

12.2 功能描述

12.3 开发指引

12.4 注意事项

12.1 概述

TCXO (温度补偿晶体振荡器 Temperature Compensated Crystal Oscillator): 是通过附加的温度补偿电路使由周围温度变化产生的振荡频率变化量削减的一种有源石英晶体振荡器, SDK 可以外接 XO 来为系统提供高精度的计数和延时功能。

TCXO 是基于外部 32MHz 晶体为系统提供高精度计数的功能。

12.2 功能描述

TCXO 模块提供的接口:

- uapi_tcxo_init: 初始化 TCXO。
- uapi_tcxo_deinit: 去初始化 TCXO。
- uapi_tcxo_get_count: 获取 TCXO 计数值。
- uapi_tcxo_get_ms: 获取 TCXO 计数毫秒值。
- uapi_tcxo_get_us: 获取 TCXO 计数微秒值。
- uapi_tcxo_delay_ms: 设置延迟毫秒数。

- uapi_tcxo_delay_us: 设置延迟微秒数。

12.3 开发指引

步骤 1 调用 uapi_tcxo_init 接口，初始化 TCXO 模块。

步骤 2 调用 uapi_tcxo_get_count 接口，获取当前 TCXO 计数值。

步骤 3 调用 uapi_tcxo_delay_ms 接口，延迟传入的时间。

----结束

示例:

```
void test_tcxo_dealysms(void)
{
    uint64_t count_before_delay_count;
    uint64_t count_after_delay_count;
    /* tcxo模块初始化 */
    uapi_tcxo_init();
    /* 通过count差值验证延迟时间 */
    count_before_delay_count = uapi_tcxo_get_count();
    uapi_tcxo_delay_ms(1000);
    count_after_delay_count= uapi_tcxo_get_count();
    printf("test case delay count %lu.\r\n",count_before_delay_count - count_after_delay_count);
    return;
}
```

12.4 注意事项

无。

13 Calendar

13.1 概述

13.2 功能描述

13.3 开发指引

13.4 注意事项

13.1 概述

Calendar 主要为系统提供基础日历功能。

13.2 功能描述

Calendar 模块提供的接口：

- uapi_calendar_init：初始化 Calendar。
- uapi_calendar_deinit：去初始化 Calendar。
- uapi_calendar_set_datetime：设置日历基线时间。
- uapi_calendar_get_datetime：获取日历当前时间。
- uapi_calendar_set_timestamp：设置基线时间戳。
- uapi_calendar_get_timestamp：获取当前时间戳。

13.3 开发指引

示例 1：设置 Calendar 时间戳，如下。

```
void test_calendar_set_stamp(void)
{
    calendar_t date;
    uapi_calendar_init();
    uapi_calendar_set_timestamp(0);/*传入0代表设置此刻为1970年1月1日0时0分0秒*/
    errcode_t ret = uapi_calendar_get_datetime(&date);
    if (ret != ERRCODE_SUCC) {
        printf("Get datetimer fail");
        return;
    }
    printf(Get datetime success: %d-%d-%d %d:%d:%d\r\n", date.year, date.mon, date.day,
    date.hour, date.min,
        date.sec);
    return;
}
```

示例 2：设置当前日期时间。

```
void test_calendar_set_date(void)
{
    calendar_t date;
    uint64_t timestamp;
    date.year = 2022;
    date.mon = 1;
    date.day = 1;
    date.hour = 12;
    date.min = 0;
    date.sec = 0;

    errcode_t ret = uapi_calendar_set_datetime(&date); /* 设置日期为当前时间 */
    if (ret != ERRCODE_SUCC) {
        printf("Set datetime fail, errcode:%u",ret);
        return ;
    }

    uapi_calendar_get_timestamp(&timestamp);/*获取2022年1月1日12时0分0秒对应的时间戳*/
    printf("now time stamp is:%llu",timestamp);
    return;
}
```

13.4 注意事项

- 日历功能约束范围为 1970 年 1 月 1 日 0 时 0 分 0 秒到 2099 年 12 月 31 日 23 时 59 分 59 秒。
- 日历时间因受片内 ULP 32k 时钟源精度影响，在 0°C ~ 55°C 温度范围内按键触发时间误差大约为：0% ~ 16% 之间。

14 Flash

14.1 概述

14.2 功能描述

14.3 开发指引

14.4 注意事项

14.1 概述

Flash 是一种非易失快闪记忆体技术，又称为闪存，支持 SPI 协议。

Flash 可以通过 SPI 协议实现读、写和擦除等多种命令，部分 Flash 支持 XIP 模式。

14.2 功能描述

Flash 模块提供的接口：

- uapi_flash_init：初始化 Flash。
- uapi_flash_deinit：去初始化 Flash。
- uapi_flash_write_data：写入数据到 Flash。
- uapi_flash_read_data：从 Flash 读取数据。
- uapi_flash_chip_erase：擦除整片 Flash 数据。
- uapi_flash_block_erase：按块擦除 Flash 数据。
- uapi_flash_sector_erase：按扇区擦除 Flash 数据。
- uapi_flash_suspend：Flash 休眠。

- uapi_flash_resume: Flash 休眠唤醒。
- uapi_flash_switch_to_deeppower: Flash 进入掉电模式。
- uapi_flash_resume_from_deeppower: Flash 掉电模式唤醒。
- uapi_flash_send_cmd_exe: 发送命令到 Flash。
- uapi_flash_get_info: 获取 Flash 当前信息。
- uapi_flash_config_cmd_at_xip_mode: 检查 Flash 是否支持连续读。
- uapi_flash_switch_to_xip_mode: Flash 进入 XIP 模式。
- uapi_flash_exit_from_xip_mode: Flash 退出 XIP 模式。
- uapi_flash_switch_to_force_bypass_mode: Flash 进入 XIP 旁路模式。
- uapi_flash_switch_to_cache_mode: Flash 进入 XIP 缓存模式。
- uapi_flash_switch_to_qspi_init: Flash 进入 QSPI 模式。
- uapi_flash_reset: Flash 复位。
- uapi_flash_read_id: Flash 读取制造 id。
- uapi_flash_read_unique_id: Flash 读取唯一 id。
- uapi_flash_read_device_id: Flash 读取设备 id。
- uapi_flash_read_status: 读 Flash 状态。

14.3 开发指引

示例:

步骤 1 首先对 Flash 做初始化, 以 GIGADEVICE_GD25LX128 为例。

```
void usr_flash_init()
{
    errcode_t errcode = uapi_flash_init(GIGADEVICE_GD25LX128);
    if (errcode != ERRCODE_SUCC) {
        printf("flash init fail\n");
    }
}
```

步骤 2 向 Flash 指定地址写数据。

```
int test_flash_write_data(unsigned int test_addr, unsigned int write_value)
{
    /* 切换到xip模式 */
}
```

```

uapi_flash_switch_to_xip_mode(GIGADEVICE_GD25LX128);
/* 读取当前地址的数据并打印出来 */

printf("addr is 0x%x, value is 0x%x, and write value is 0x%x", test_addr,
       readl(test_addr + FLASH_START), write_value);

/* 退出xip模式 */

uapi_flash_exit_from_xip_mode(GIGADEVICE_GD25LX128);
/* 删除指定地址数据 */

uapi_flash_sector_erase(GIGADEVICE_GD25LX128, test_addr, false);
/* 写数据到指定地址 */

uapi_flash_write_data(GIGADEVICE_GD25LX128, test_addr, (uint8_t *)&write_value,
sizeof(write_value));

/* 切换回xip模式 */

uapi_flash_switch_to_xip_mode(GIGADEVICE_GD25LX128);
/* 打印出当前地址数据 */

printf("addr is 0x%x, value is 0x%x.", test_addr, readl(test_addr + FLASH_START));
return 0;
}

```

步骤 3 从 Flash 指定地址读取数据。

```

int test_flash_read_data(uint32_t test_addr)
{
    unsigned int dst[10] = { 0 };
    /* 退出xip模式 */

    uapi_flash_exit_from_xip_mode(GIGADEVICE_GD25LX128);
    /* 读取test_addr的flash数据到指定地址 */

    uapi_flash_read_data(GIGADEVICE_GD25LX128, test_addr, (uint8_t*)dst, sizeof(dst));
    for (int i = 0; i < 10; i++) {
        printf("dst = 0x%x\r\n", dst[i]);
    }

    /* 切换回xip模式 */

    uapi_flash_switch_to_xip_mode(GIGADEVICE_GD25LX128);
    return 0;
}

```

步骤 4 从 Flash 指定块擦除数据。

```

int test_flash_erase_data(uint32_t test_addr, uint32_t test_len)
{
    unsigned int dst[10] = { 0 };

```



```
/* 退出xip模式 */  
uapi_flash_exit_from_xip_mode(GIGADEVICE_GD25LX128);  
/* 擦除test_addr的flash数据 */  
uapi_flash_block_erase(GIGADEVICE_GD25LX128, test_addr, test_len, true);  
/* 切换回xip模式 */  
uapi_flash_switch_to_xip_mode(GIGADEVICE_GD25LX128);  
return 0;  
}
```

----结束

14.4 注意事项

如果 XIP 缓存模式使能场景，对于 Flash 进行读写擦除前，务必请退出 XIP 缓存模式，操作完成后请再切换回 XIP 缓存模式。

15 Key

15.1 概述

15.2 功能描述

15.3 开发指引

15.4 注意事项

15.1 概述

Key 即按键，分为两种类型：

- Power Key 按键，默认支持。
- Function Key 按键，可通过 GPIO 外接按键实现，最多可扩展 7 个 Function 按键。

两种按键可以同时支持使用，目前按键支持上报单击、双击、短按、长按四种按键事件。

15.2 功能描述

button 模块提供的接口：

- uapi_button_init：初始化按键。
- uapi_button_deinit：去初始化按键。
- uapi_button_register：注册按键。
- uapi_button_deregister：去注册按键。
- uapi_button_gpio_register：注册按键回调。

- uapi_button_send_msg_register: 发送函数注册接口。
- uapi_power_set_mode: 配置 Power Key 按键事件类型 (重启/关机)。
- uapi_power_time_mode: 配置 Power Key 按键事件 (重启/关机) 的触发时间。

15.3 开发指引

示例:

步骤 1 按键初始化。

```
errcode_t uapi_button_init(void)
{
    errcode_t ret = 0;
    ret = uapi_timer_init();
    ret |= uapi_timer_adapter(1, TIMER_1_IRQN, irq_prio(TIMER_1_IRQN)); /* 1: using timer1 */
    if (ret != ERRCODE_SUCC) {
        PRINT("uapi_timer_init FAILED 0x%x", ret);
        return ret;
    }
    ret = uapi_timer_create(1, &g_button_timer); /* 1: using timer1 */
    if (ret != ERRCODE_SUCC) {
        PRINT("uapi_timer_create FAILED 0x%x", ret);
        return ret;
    }
    return ERRCODE_SUCC;
}
```

步骤 2 注册按键引脚映射。

```
errcode_t uapi_button_register(button_map_t* button_map)
{
    errcode_t ret = 0;
    if (button_map == NULL) {
        return ERRCODE_FAIL;
    }
    button_peripheral_api *button_api = button_port_get_api();
    for (uint8_t i = 0; i < BUTTON_PIN_MAP_MAX; i++) {
        g_button_pin_map[i].pin = button_map[i].pin;
        g_button_pin_map[i].button = button_map[i].button;
        if (button_map[i].pin != PIN_NONE && button_map[i].button != BUTTON_MAX) {
            if (button_map[i].button == BUTTON_PWR) {
                button_api->button_pmu_pwr_cfg(true);
            }
        }
    }
}
```

```

    }
    ret |= uapi_button_gpio_register(g_button_pin_map[i].pin);
} else if (button_map[i].pin == PIN_NONE && button_map[i].button == BUTTON_PWR) {
    ret = button_api->button_register_callback(g_button_pin_map[i].pin,
(osal_irq_handler)button_irq_callback);
    if (ret != ERRCODE_SUCC) {
        PRINT("button register failed\r\n");
        return ret;
    }
    continue;
}
}
return ret;
}

```

步骤3 注册发送函数。

```

void uapi_button_send_msg_register(msg_send_callback_t cb)
{
    if (cb != NULL) {
        g_msg_send_cb = cb;
    }
}

```

----结束

15.4 注意事项

- uapi_power_set_mode 接口支持设置 Power Key 强制触发事件，目前支持重启和关机两种事件可配置，默认为重启。

参数如下：

```

typedef enum {
    PDMODE_RE_UP,    // 重启
    PDMODE_DOWN,    // 关机
    PDMODE_MAX
} power_mode_t;

```

- uapi_power_time_mode 接口支持设置 Power Key 强制触发事件的时间阈值，目前支持 Level0、Level1、Level2 和 Level3 四档时间设置，默认为 Level1。

参数如下：

```

typedef enum {

```

```
PDTIME_LEVEL0,  
PDTIME_LEVEL1,  
PDTIME_LEVEL2,  
PDTIME_LEVEL3,  
PDTIME_LEVELMAX  
} power_time_t;
```

其中 Level0、Level1、Level2 和 Level3 分别代表设置时间阈值为：10s、11s、12s、13s。

该设置时间因受片内 ULP 32K 时钟源精度影响，在 0°C ~ 55°C 温度范围内按键触发时间误差大约为：0% ~ 16% 之间。

16 SDIO

16.1 概述

16.2 功能描述

16.3 开发指引

16.4 注意事项

16.1 概述

SDIO (Secure Digital Input and Output 安全数字输入输出) 在 SD 标准上定义了一种外设接口。

16.2 功能描述

SDIO 模块提供的接口：

- MMC_HostInitById：SDIO 驱动初始化。
- sdio_get_func：获得一个 sdio_func，此 sdio_func 在 SDIO 设备识别后产生，通过 func 号、厂商 ID、设备 ID（此三个参数根据 SDIO 外设获得）唯一标识。
- sdio_en_func：使能 SDIO，该接口通过 CMD52 命令写 0x2 地址实现。
- sdio_dis_func：禁止指定的 sdio_func，该接口通过 CMD52 命令写 0x2 地址实现。
- sdio_require_irq：给指定的 sdio_func 注册 SDIO 中断处理函数，该中断处理函数需要用户自行实现。
- sdio_release_irq：释放指定 sdio_func 中注册的 SDIO 中断处理函数。

- `sdio_read_byte`: 从指定 `sdio_func` 的指定地址 `addr` 读取一字节数据, 并返回读取的数据。
- `sdio_read_incr_block`: 从指定 `sdio_func` 的指定地址 `addr` 读取指定长度的数据到内存指定地址 `dst` 中, 读取时设备地址会依次增长 (比如设备的内存地址) 该接口通过 `CMD53` 命令实现。
- `sdio_read_fifo_block`: 从指定 `sdio_func` 的指定地址 `addr` 读取指定长度的数据到内存指定地址 `dst` 中, 读取时设备地址固定不变 (比如设备的 FIFO), 该接口通过 `CMD53` 命令实现。
- `sdio_write_incr_block`: 将指定地址 `src` 中 `size` 长度的数据写入指定 `sdio_func` 的指定地址 `addr` 中, 写入时设备地址会依次增长 (比如设备的内存地址), 该接口通过 `CMD53` 命令实现。
- `sdio_write_fifo_block`: 将指定地址 `src` 中 `size` 长度的数据写入指定 `sdio_func` 的指定地址 `addr` 中, 写入时设备地址固定不变 (比如设备的 FIFO), 该接口通过 `CMD53` 命令实现。
- `sdio_write_byte`: 将一字节数据写入指定 `sdio_func` 的指定地址 `addr`, 该接口通过 `CMD52` 命令实现。
- `sdio_write_byte_raw`: 将一字节数据写入指定 `sdio_func` 的指定地址 `addr`, 写完后读回 (设置 `RAW Flag`), 该接口通过 `CMD52` 命令实现。
- `sdio_func0_read_byte`: 从设备指定地址 `addr` 读取一字节数据, 并返回读取的数据, 该接口通过 `CMD52` 命令读取 `func0` 实现。
- `sdio_func0_write_byte`: 写入一字节数据到设备指定地址 `addr`, 该接口通过 `CMD52` 命令写入 `func0` 实现。
- `sdio_set_cur_blk_size`: 配置 SDIO 当前块大小, 此块大小不应大于 512Byte。
- `sdio_rescan`: SDIO 设备重新识别, 实际执行操作为设备卸载和加载。
- `sdio_reset_comm`: 复位 SDIO 设备。

16.3 开发指引

步骤 1 管脚复用: 配置 IO 寄存器, 配置对应 SDIO 外围设备的 CLK、CMD、DATA0、DATA1、DATA2、DATA3 管脚寄存器。

步骤 2 初始化驱动: 调用 `MMC_HostInitById` 接口。

步骤 3 设备识别后获取 `func` 并使能 `func`。

```
struct sdio_func* func;
```

```
func = sdio_get_func(func_num, manf_id, device_id);  
sdio_en_func(func);
```

步骤 4 设置当前 block size：调用 sdio_set_cur_blk_size 接口。

步骤 5 注册中断处理函数。

```
void handler_sample(struct sdio_func *func)  
{  
    dprintf( "%s,%d func:0x%x\n", __FUNCTION__, __LINE__, func);  
    return;  
}  
sdio_require_irq(func, handler_sample);
```

步骤 6 通过 SDIO 对外读写接口进行读写等操作。

----结束

16.4 注意事项

无。